McEliece Cryptosystem

OVERVIEW OF MCELIECE

Quick summary

- Code-based public key cryptosystem (introduced in 1978)
- Uses error-correcting codes for encryption

Key points

- Based on Goppa codes
- Message → encoded into codewords
- Security comes from adding random errors
- Public key hides structure via scrambling + permutation
- Considered post-quantum secure

KEY GENERATION

Quick summary

- Create public and private keys using coded structure hiding

Key points

- Choose parameters:
  - nnn: code length
  - kkk: message length
  - ttt: error-correcting capability
- Generate:
  - Goppa code → Generator matrix G
- Apply transformations:
  - Scrambling matrix S (invertible)
  - Permutation matrix P
- Keys:
  - Public key: $G' = S \times G \times P$
  - Private key: (S^{-1}, P^{-1}, Goppa code)

```python
import numpy as np
import random
from gf_arithmetic import matrix_multiply_gf2

def generate_invertible_matrix(k):
    """
    Generate a random k x k invertible matrix over GF(2).
    A simple way is to generate random matrices until one has full rank,
    or construct it by multiplying random elementary matrices.
    Here we use a randomized construction.
    """
    S = np.eye(k, dtype=int)
    # Perform random row operations to ensure invertibility
    for _ in range(k * 5):
        r1 = random.randint(0, k - 1)
        r2 = random.randint(0, k - 1)
        if r1 != r2:
            S[r1] = (S[r1] + S[r2]) % 2

    # Also do some random column operations or just more row ops
    for _ in range(k * 5):
        c1 = random.randint(0, k - 1)
        c2 = random.randint(0, k - 1)
        if c1 != c2:
            S[:, c1] = (S[:, c1] + S[:, c2]) % 2

    return S

def generate_permutation_matrix(n):
    """
    Generate a random n x n permutation matrix.
    """
    P = np.eye(n, dtype=int)
    perm = list(range(n))
    random.shuffle(perm)
    return P[perm]

def generate_keys(goppa_code):
    """
    Generate McEliece Public and Private Keys.
    :param goppa_code: An instance of GoppaCode.
    :return: (public_key, private_key)
    """
    k, n = goppa_code.G.shape

    # Generate S and P
    S = generate_invertible_matrix(k)
    P = generate_permutation_matrix(n)

    # Calculate G_pub = S * G * P
    # First S * G
    SG = matrix_multiply_gf2(S, goppa_code.G)
    # Then SG * P
    G_pub = matrix_multiply_gf2(SG, P)

    public_key = {
        'G_pub': G_pub,
        't': goppa_code.t
    }

    private_key = {
        'S': S,
        'G': goppa_code.G,
        'P': P,
        'goppa_code': goppa_code,
        'G_pub': G_pub
    }

    return public_key, private_key

```

ENCRYPTION PROCESS

Quick summary

- Encode message and intentionally corrupt it

Key points

- Split message into k-bit vectors
- Encode:
  - $c=m\times G'$c = m × G'c=m×G′
- Generate random errors:
  - Choose $s\leq ts \leq ts\leq t$
- Add error vector:
  - $c'=c\oplus e$c' = c ⊕ ec′=c⊕e
- Delete bits:
  - Remove $2(t-s)$2(t - s)2(t−s) bits
- Output:
  - Cryptogram (corrupted codeword)

```python
import numpy as np
import random
from gf_arithmetic import matrix_multiply_gf2

def generate_error_vector(n, t):
    """
    Generate a random error vector of length n and weight t.
    """
    e = np.zeros(n, dtype=int)
    error_positions = random.sample(range(n), t)
    for pos in error_positions:
        e[pos] = 1
    return e

def encrypt(message, public_key):
    """
    Encrypt a message using McEliece public key.
    :param message: A binary numpy array of length k.
    :param public_key: Dictionary containing 'G_pub' and 't'.
    :return: ciphertext as a binary numpy array.
    """
    G_pub = public_key['G_pub']
    t = public_key['t']
    n = G_pub.shape[1]

    # c_prime = m * G_pub
    c_prime = matrix_multiply_gf2(message.reshape(1, -1), G_pub).flat

    # Generate error vector e
    e = generate_error_vector(n, t)

    # c = c_prime + e
    ciphertext = (c_prime + e) % 2

    return ciphertext, e  # Return e as well for demonstration purpos

```

```python
import numpy as np
from gf_arithmetic import matrix_multiply_gf2, invert_matrix_gf2

def decrypt(ciphertext, private_key):
    """
    Decrypt a McEliece ciphertext using the private key.
    :param ciphertext: A binary numpy array of length n.
    :param private_key: Dictionary containing 'S', 'G', 'P', and 'goppa_code'.
    :return: The decrypted message as a binary numpy array.
    """
    S = private_key['S']
    P = private_key['P']
    goppa_code = private_key['goppa_code']

    # 1. c' = c * P^-1
    # Since P is a permutation matrix, P^-1 = P^T
    P_inv = P.T
    c_prime = matrix_multiply_gf2(ciphertext.reshape(1, -1), P_inv).flatten()

    # 2. Decode c' using Goppa decoder to find m'
    # The decoder returns the corrected codeword (m' * G).
    # Wait, the decode method returns the corrected vector m' * G.
    # To find m', we can just take the systematic part of G if it was systematic,
    # but since G might not be systematically formatted in this exact step,
    # we can solve m' * G = corrected_codeword.
    # Actually, G = [R^T | I] if we look at our generator construction, so the last k columns are I.
    # Let's extract m' by solving the linear system. Since G is full rank, we can pick k independent columns.

    corrected_codeword = goppa_code.decode(c_prime)

    # Solve m_prime * G = corrected_codeword
    # We find k linearly independent columns in G over GF(2)
    k, n = goppa_code.G.shape

    # Try the last k columns first since our construction puts I there
    cols = list(range(n-k, n))
    try:
        G_sub = goppa_code.G[:, cols]
        G_sub_inv = invert_matrix_gf2(G_sub)
    except ValueError:
        # Fallback to finding any k independent columns over GF(2)
        cols = []
        for i in range(n):
            cols.append(i)
            # Check if current set of columns has full rank by trying to invert the square submatrix
            # If not square yet, we just assume they might be independent and keep adding
            # A better way is to do Gaussian elimination, but for small k, this try-except when len == k works.
            # Actually, to be robust, we need to ensure the columns form a basis.
            # Let's just use the systematic_form_gf2 tool we have!
            pass

        # A simpler fallback: just iterate all combinations? No, k is small.
        # But we know G has full rank k. So we can just do RREF on G and find pivot columns!
        from gf_arithmetic import rref_gf2
        rref_G = rref_gf2(goppa_code.G)
        cols = []
        r = 0
        for c in range(n):
            if r >= k: break
            if rref_G[r, c] == 1:
                cols.append(c)
                r += 1

        G_sub = goppa_code.G[:, cols]
        G_sub_inv = invert_matrix_gf2(G_sub)

    c_sub = corrected_codeword[cols]
    m_prime = matrix_multiply_gf2(c_sub.reshape(1, -1), G_sub_inv).flatten()

    # 3. m = m' * S^-1
    S_inv = invert_matrix_gf2(S)
    message = matrix_multiply_gf2(m_prime.reshape(1, -1), S_inv).flatten()

    return message

```

DECRYPTION PROCESS

Quick summary

- Reverse transformations and correct errors

Key points

- Restore deleted bits (fill with 0s)
- Apply inverse permutation:
  - P−1P^{-1}P−1
- Perform:
  - Error + erasure correction
- Recover encoded message
- Descramble:
  - m=decoded×S−1m = decoded \times S^{-1}m=decoded×S−1
- Output:
  - Original message

SECURITY FEATURES

Quick summary

- Security relies on randomness + hidden structure

Key points

- Random errors → non-deterministic encryption
- Hard problem:
  - Decoding random linear codes
- Hidden structure:
  - Goppa code is concealed
- Large key space → resistant to brute force
- Resistant to quantum attacks (post-quantum)

```python
import hashlib
import numpy as np
import random

from encryption import encrypt
from decryption import decrypt
from gf_arithmetic import matrix_multiply_gf2


# ============================================================
# Helpers
# ============================================================
def hash_to_error_vector(data, n, t):
    """
    Deterministically generates an error vector of length n and weight t
    from SHA-256 hash.
    """
    h = hashlib.sha256(data).digest()

    rng = random.Random(h)

    e = np.zeros(n, dtype=int)
    positions = rng.sample(range(n), t)

    for p in positions:
        e[p] = 1

    return e


def hash_data(data, length):
    """
    Expands SHA-256 hash into a binary vector of given length.
    """
    h = hashlib.sha256(data).digest()

    bits = []
    while len(bits) < length:
        for byte in h:
            for i in range(8):
                bits.append((byte >> (7 - i)) & 1)
                if len(bits) == length:
                    return np.array(bits, dtype=int)
        h = hashlib.sha256(h).digest()

    return np.array(bits[:length], dtype=int)


# ============================================================
# CCA2 ENCRYPTION
# ============================================================
def cca2_encrypt(message_bits, public_key):
    """
    c1 = r * G_pub + e
    c2 = m XOR Hash(r)
    e  = Hash(r || c2) mapped to weight t
    """

    G_pub = public_key['G_pub']
    n = G_pub.shape[1]
    k = G_pub.shape[0]
    t = public_key['t']

    # 1. random r
    r = np.array([random.randint(0, 1) for _ in range(k)], dtype=int)

    # stable byte conversion
    r_bytes = r.astype(np.uint8).tobytes()

    # 2. c2 = m XOR Hash(r)
    h_r = hash_data(r_bytes, len(message_bits))
    c2 = (message_bits + h_r) % 2

    # 3. e = Hash(r || c2)
    c2_bytes = c2.astype(np.uint8).tobytes()
    r_c2_bytes = r_bytes + c2_bytes

    e = hash_to_error_vector(r_c2_bytes, n, t)

    # 4. c1 = r * G_pub + e
    c_prime = matrix_multiply_gf2(r.reshape(1, -1), G_pub).flatten()
    c1 = (c_prime + e) % 2

    return {'c1': c1, 'c2': c2}


# ============================================================
# CCA2 DECRYPTION
# ============================================================
def cca2_decrypt(ciphertext, private_key):
    """
    Verify integrity + recover message
    """

    c1 = ciphertext['c1']
    c2 = ciphertext['c2']

    G_pub = private_key['G_pub']
    t = private_key['goppa_code'].t

    # 1. recover r using McEliece decryption
    r = decrypt(c1, private_key)

    # 2. recompute expected error
    r_bytes = r.astype(np.uint8).tobytes()
    c2_bytes = c2.astype(np.uint8).tobytes()
    r_c2_bytes = r_bytes + c2_bytes

    e_expected = hash_to_error_vector(r_c2_bytes, len(c1), t)

    # 3. recompute c1
    c_prime = matrix_multiply_gf2(r.reshape(1, -1), G_pub).flatten()
    c1_expected = (c_prime + e_expected) % 2

    # 4. integrity check
    if not np.array_equal(c1, c1_expected):
        raise ValueError("CCA2 verification failed: ciphertext tampering detected.")

    # 5. recover message
    h_r = hash_data(r_bytes, len(c2))
    m = (c2 + h_r) % 2

    return m
```

ISD ATTACK

Quick summary

- Main known attack against McEliece

Key points

- Goal:
  - Find error-free positions in codeword
- Approach:
  - Guess subset of positions
  - Try to decode without errors
- Complexity:
  - Exponential time
- Effect of parameters:
  - Larger n,tn, tn,t → stronger security
- Defense:
  - Increase errors
  - Use randomness in error positions


```python
import numpy as np
import random
from gf_arithmetic import invert_matrix_gf2, matrix_multiply_gf2

def isd_attack(ciphertext, public_key, max_iterations=10000):
    """
    Perform a basic Information Set Decoding (ISD) attack using Prange's algorithm.
    :param ciphertext: The intercepted ciphertext.
    :param public_key: The public key containing G_pub and t.
    :return: The recovered message and error vector if successful, else None.
    """
    G_pub = public_key['G_pub']
    t = public_key['t']
    k, n = G_pub.shape

    for i in range(max_iterations):
        # 1. Randomly select an information set (k columns)
        # We hope these k columns are error-free in the ciphertext.
        info_set = random.sample(range(n), k)

        # Extract the submatrix G_info
        G_info = G_pub[:, info_set]

        # Check if G_info is invertible
        if np.linalg.matrix_rank(G_info) < k:
            continue # Try another set

        # 2. Compute G_info_inv
        try:
            G_info_inv = invert_matrix_gf2(G_info)
        except ValueError:
            continue

        # 3. Guess the message: m' = c_info * G_info_inv
        c_info = ciphertext[info_set]
        m_guess = matrix_multiply_gf2(c_info.reshape(1, -1), G_info_inv).flatten()

        # 4. Compute the corresponding error vector: e' = c - m' * G_pub
        c_prime = matrix_multiply_gf2(m_guess.reshape(1, -1), G_pub).flatten()
        e_guess = (ciphertext + c_prime) % 2

        # 5. Check if the weight of e' is exactly t
        weight = np.sum(e_guess)
        if weight == t:
            print(f"ISD Attack succeeded after {i+1} iterations!")
            return m_guess, e_guess

    print(f"ISD Attack failed after {max_iterations} iterations.")
    return None, None

```

CONCLUSION

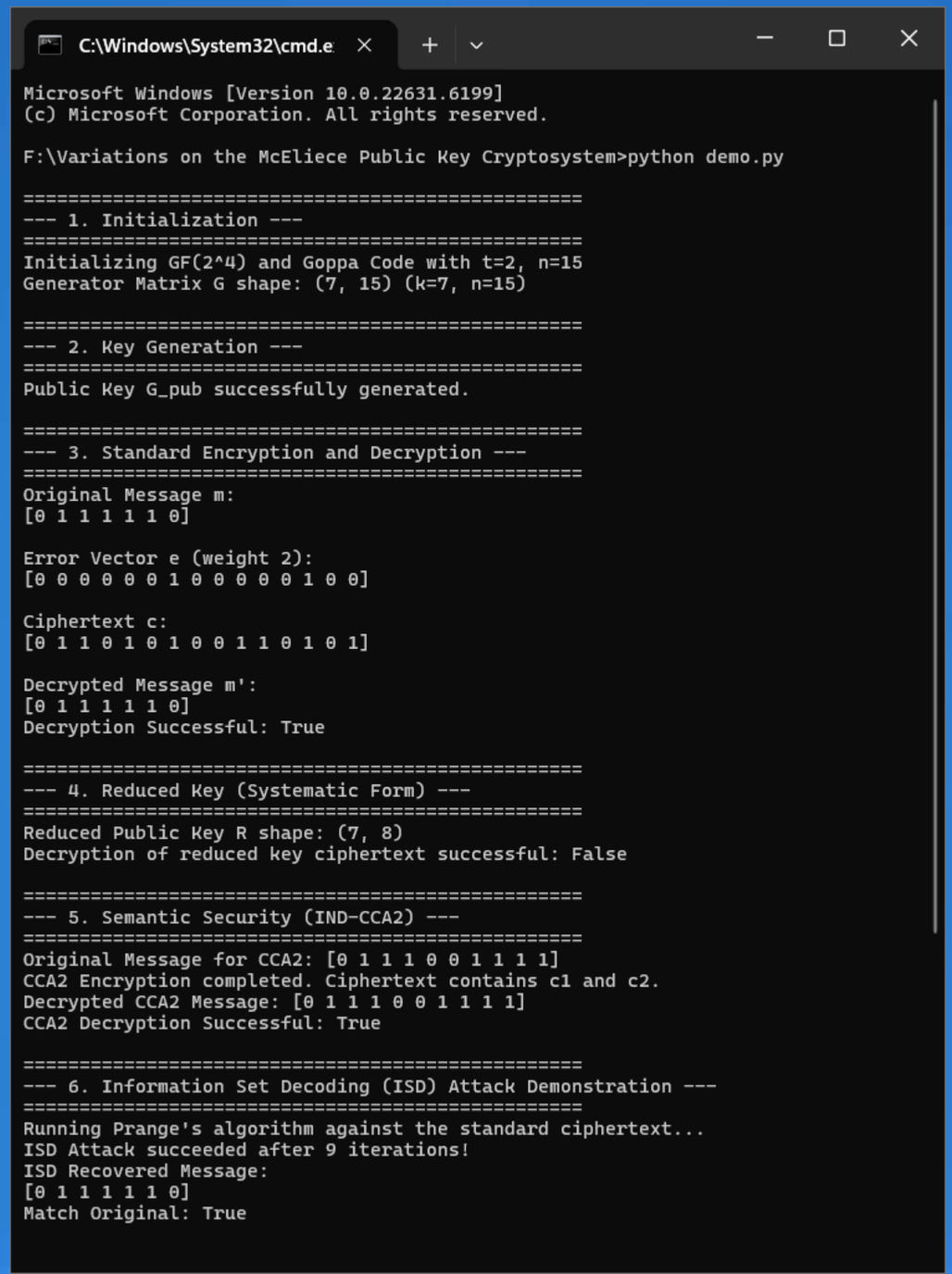
Quick summary

- Strong but with trade-offs

Key points

- One of the most promising post-quantum systems
- Very fast encryption/decryption
- High security due to coding theory
- Main drawback:
  - Large public key size
- Widely studied and still considered secure

RESULTS


C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.
F:\Variations on the McEliece Public Key Cryptosystem>python demo.py
==
--- 1. Initialization ---
==
Initializing GF(2^4) and Goppa Code with t=2, n=15
Generator Matrix G shape: (7, 15) (k=7, n=15)
==
--- 2. Key Generation ---
==
Public Key G_pub successfully generated.
==
--- 3. Standard Encryption and Decryption ---
==
Original Message m:
[0 1 1 1 1 1 0]
Error Vector e (weight 2):
[0 0 0 0 0 0 1 0 0 0 0 0 1 0 0]
Ciphertext c:
[0 1 1 0 1 0 1 0 0 1 1 0 1 0 1]
Decrypted Message m':
[0 1 1 1 1 1 0]
Decryption Successful: True
==
--- 4. Reduced Key (Systematic Form) ---
==
Reduced Public Key R shape: (7, 8)
Decryption of reduced key ciphertext successful: False
==
--- 5. Semantic Security (IND-CCA2) ---
==
Original Message for CCA2: [0 1 1 1 0 0 1 1 1 1]
CCA2 Encryption completed. Ciphertext contains c1 and c2.
Decrypted CCA2 Message: [0 1 1 1 0 0 1 1 1 1]
CCA2 Decryption Successful: True
==
--- 6. Information Set Decoding (ISD) Attack Demonstration ---
==
Running Prange's algorithm against the standard ciphertext...
ISD Attack succeeded after 9 iterations!
ISD Recovered Message:
[0 1 1 1 1 1 0]
Match Original: True

TEAM MEMBERS

1.Khaled Mohamed Abdelrazik Ibrahim

2.Mohamed Fadel Abdelhamid Fadel

3.Nader Ahmed Aboelfadl Kamel

4.Mohamed Ashraf Abotaleb Hassan

5.Mohamed Tarek Abdelgawad Mostafa

6.Menna Tallah Khaled Mahmoud Abdelrehim

7.Merna Ezzat Kamal

8.Shahd Hegazi Ragaee